

# Quantum Kernels and SVMs

## QML

---

### Contents

|     |                                         |    |
|-----|-----------------------------------------|----|
| 1   | Overview                                | 3  |
| 2   | QSVC: Quantum Support Vector Classifier | 3  |
| 2.1 | The Optimization Problem                | 3  |
| 2.2 | The KKT Conditions                      | 4  |
| 2.3 | Decision Function                       | 4  |
| 2.4 | Multi-Class Extension                   | 5  |
| 3   | QSVR: Quantum Support Vector Regressor  | 5  |
| 3.1 | The $\epsilon$ -Insensitive Loss        | 5  |
| 3.2 | QSVR Optimization                       | 5  |
| 3.3 | Choosing $\epsilon$ and $C$             | 6  |
| 4   | Pegasos QSVC                            | 6  |
| 4.1 | The Pegasos Algorithm                   | 7  |
| 4.2 | Pegasos vs Standard SVM                 | 7  |
| 4.3 | When to Use Pegasos QSVC                | 7  |
| 4.4 | Convergence Analysis                    | 8  |
| 5   | Kernel Matrix Analysis                  | 8  |
| 5.1 | Eigenvalue Decomposition                | 8  |
| 5.2 | What the Spectrum Tells U               | 9  |
| 5.3 | Connection to SVM Performance           | 9  |
| 6   | Training on Real Datasets               | 10 |
| 6.1 | Iris Dataset                            | 10 |
| 6.2 | Wine Dataset                            | 10 |
| 6.3 | Breast Cancer Dataset                   | 11 |
| 7   | Cross-Validation with Quantum Kernels   | 11 |
| 7.1 | The Cost Problem                        | 11 |
| 7.2 | Smart Cross-Validation                  | 12 |
| 7.3 | Nested Cross-Validation                 | 12 |

|      |                                                          |    |
|------|----------------------------------------------------------|----|
| 8    | Hyperparameter Tuning                                    | 13 |
| 8.1  | Feature Map Parameters . . . . .                         | 13 |
| 8.2  | SVM Hyperparameters . . . . .                            | 13 |
| 8.3  | Tuning Strategy . . . . .                                | 13 |
| 9    | Computational Cost Analysis                              | 14 |
| 9.1  | Kernel Matrix Construction . . . . .                     | 14 |
| 9.2  | Prediction Cost . . . . .                                | 15 |
| 9.3  | Comparison with Classical Kernels . . . . .              | 15 |
| 9.4  | Total Budget Planning . . . . .                          | 15 |
| 10   | Classical vs Quantum Kernel Comparison                   | 16 |
| 10.1 | When Quantum Kernels Can Win . . . . .                   | 16 |
| 10.2 | Empirical Comparison: Feature Map Expressivity . . . . . | 16 |
| 10.3 | When Quantum Doesn't Win . . . . .                       | 16 |
| 11   | Limitations and When NOT to Use Quantum Kernels          | 17 |
| 11.1 | Dataset Size Limits . . . . .                            | 17 |
| 11.2 | Noise Sensitivity . . . . .                              | 17 |
| 11.3 | The Concentration Problem . . . . .                      | 18 |
| 11.4 | When NOT to Use Quantum Kernels . . . . .                | 18 |
| 11.5 | Decision Flowchart . . . . .                             | 19 |
| 12   | Advanced Topics                                          | 19 |
| 12.1 | Trainable Quantum Kernels . . . . .                      | 19 |
| 12.2 | Quantum Kernel Bandwidth . . . . .                       | 20 |
| 12.3 | Kernel Principal Component Analysis . . . . .            | 20 |
| 13   | Putting It All Together                                  | 20 |
| 14   | Summary and Key Takeaways                                | 21 |

# 1 Overview

last lecture we built the practical pipeline for quantum kernels — from qiskit implementation to benchmarking on toy datasets. now we go deeper into the SVM side of things and tackle the real challenges: training on actual datasets, cross-validation, hyperparameter tuning, and understanding when quantum kernels r worth the effort.

this lecture covers:

- QSVC in detail — the complete classifier pipeline with all the math
- QSVR — regression with quantum kernels
- pegasos QSVC — a faster online training algorithm
- kernel matrix analysis — eigenvalue spectra and wt they tell u
- training on real datasets: iris, wine, breast cancer
- cross-validation with expensive quantum kernels
- hyperparameter tuning strategies
- computational cost analysis
- comparison with classical kernels — when quantum wins and when it doesnt
- limitations and when NOT to use quantum kernels

tbh this is the “reality check” lecture. we r going to be honest about wt works, wt doesnt, and wt the actual computational costs r.

## 2 QSVC: Quantum Support Vector Classifier

lets start with the full mathematical treatment of QSVC. this is just a classical SVM with a quantum kernel, but the details matter.

### 2.1 The Optimization Problem

given training data  $\{(\mathbf{x}_i, y_i)\}_{i=1}^M$  with  $y_i \in \{-1, +1\}$  and a quantum kernel  $\kappa(\mathbf{x}, \mathbf{x}') = |\langle \phi(\mathbf{x}) | \phi(\mathbf{x}') \rangle|^2$ , the SVM primal problem is:

Definition 2.1 (QSVC Primal Problem).

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^M \xi_i$$

subject to:

$$y_i(\mathbf{w} \cdot \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad i = 1, \dots, M \quad (1)$$

$$\xi_i \geq 0, \quad i = 1, \dots, M \quad (2)$$

where  $\phi(\mathbf{x})$  is the quantum feature map,  $\mathbf{w}$  lives in the quantum feature space (dimension  $2^n$ ), and  $C > 0$  is the regularization parameter.

we never actually compute  $\mathbf{w}$  in the  $2^n$ -dimensional space. instead, we solve the dual:

Definition 2.2 (QSVC Dual Problem).

$$\max_{\alpha} \sum_{i=1}^M \alpha_i - \frac{1}{2} \sum_{i,j=1}^M \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)$$

subject to:

$$0 \leq \alpha_i \leq C, \quad i = 1, \dots, M \quad (3)$$

$$\sum_{i=1}^M \alpha_i y_i = 0 \quad (4)$$

in matrix form, using the kernel matrix  $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ :

$$\max_{\alpha} \mathbf{1}^\top \alpha - \frac{1}{2} \alpha^\top (\mathbf{Y} \mathbf{K} \mathbf{Y}) \alpha$$

where  $\mathbf{Y} = \text{diag}(y_1, \dots, y_M)$ .

## 2.2 The KKT Conditions

the karush-kuhn-tucker conditions for the SVM r important bcs they tell us wt support vectors r:

Theorem 2.1 (KKT Conditions for QSVC). at the optimal solution  $(\alpha^*, b^*)$ :

1. if  $\alpha_i^* = 0$ : point  $\mathbf{x}_i$  is correctly classified with margin  $\geq 1$
2. if  $0 < \alpha_i^* < C$ : point  $\mathbf{x}_i$  is exactly on the margin ( $y_i f(\mathbf{x}_i) = 1$ ). these r the support vectors.
3. if  $\alpha_i^* = C$ : point  $\mathbf{x}_i$  is a margin violator (misclassified or inside margin)

the bias  $b^*$  is computed from any support vector with  $0 < \alpha_i^* < C$ :

$$b^* = y_i - \sum_{j=1}^M \alpha_j^* y_j \kappa(\mathbf{x}_j, \mathbf{x}_i)$$

## 2.3 Decision Function

once trained, the QSVC classifies new points as:

$$\hat{y} = \text{sign} \left( \sum_{i \in \mathcal{SV}} \alpha_i^* y_i \kappa(\mathbf{x}, \mathbf{x}_i) + b^* \right)$$

where  $\mathcal{SV} = \{i : \alpha_i^* > 0\}$  is the set of support vectors. note that for prediction, u only need to compute kernel values between the new point and the support vectors, not all training points.

Key Point. the number of support vectors directly affects prediction cost. for each new test point, u need  $|\mathcal{SV}|$  kernel evaluations, each requiring a quantum circuit execution. if the SVM has many support vectors (which happens with noisy kernels), prediction is expensive.

## 2.4 Multi-Class Extension

QSVC handles multi-class problems using one-vs-rest (OVR) or one-vs-one (OVO):

one-vs-rest: for  $K$  classes, train  $K$  binary classifiers. classifier  $k$  separates class  $k$  from all others. predict the class with highest decision function value:

$$\hat{y} = \arg \max_k f_k(\mathbf{x})$$

one-vs-one: train  $\binom{K}{2}$  binary classifiers, one for each pair of classes. predict by majority vote.

Intuition. for quantum kernels, one-vs-one is usually better bcs each binary problem uses a smaller subset of the data, which means smaller kernel matrices. the downside is more classifiers to train. for  $K = 3$  classes: OVR needs 3 classifiers but each uses all  $M$  points. OVO needs 3 classifiers too but each uses roughly  $2M/3$  points.

## 3 QSVR: Quantum Support Vector Regressor

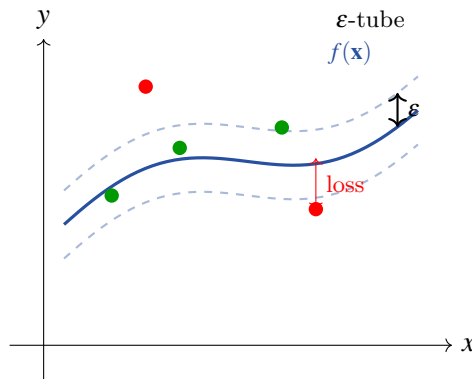
QSVR extends the quantum kernel approach to regression. instead of classifying, we predict continuous values.

### 3.1 The $\epsilon$ -Insensitive Loss

Definition 3.1 ( $\epsilon$ -Insensitive Loss). the  $\epsilon$ -insensitive loss function is:

$$L_\epsilon(y, f(\mathbf{x})) = \max(0, |y - f(\mathbf{x})| - \epsilon) = \begin{cases} 0 & \text{if } |y - f(\mathbf{x})| \leq \epsilon \\ |y - f(\mathbf{x})| - \epsilon & \text{otherwise} \end{cases}$$

this means we dont penalize errors smaller than  $\epsilon$ . the regression function has an “ $\epsilon$ -tube” around it where errors r tolerated.



### 3.2 QSVR Optimization

Definition 3.2 (QSVR Primal Problem).

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^M (\xi_i + \xi_i^*)$$

subject to:

$$y_i - \mathbf{w} \cdot \phi(\mathbf{x}_i) - b \leq \varepsilon + \xi_i \quad (5)$$

$$\mathbf{w} \cdot \phi(\mathbf{x}_i) + b - y_i \leq \varepsilon + \xi_i^* \quad (6)$$

$$\xi_i, \xi_i^* \geq 0 \quad (7)$$

the dual problem introduces two sets of lagrange multipliers  $\alpha_i$  and  $\alpha_i^*$ :

Definition 3.3 (QSVR Dual Problem).

$$\max_{\alpha, \alpha^*} -\frac{1}{2} \sum_{i,j=1}^M (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) K_{ij} - \varepsilon \sum_{i=1}^M (\alpha_i + \alpha_i^*) + \sum_{i=1}^M y_i (\alpha_i - \alpha_i^*)$$

subject to:

$$\sum_{i=1}^M (\alpha_i - \alpha_i^*) = 0 \quad (8)$$

$$0 \leq \alpha_i, \alpha_i^* \leq C \quad (9)$$

the regression function:

$$f(\mathbf{x}) = \sum_{i=1}^M (\alpha_i^* - \alpha_i) \kappa(\mathbf{x}, \mathbf{x}_i) + b$$

Key Point. in QSVR, support vectors  $\mathbf{r}$  points that lie on or outside the  $\varepsilon$ -tube. points inside the tube have  $\alpha_i = \alpha_i^* = 0$  and dont contribute to the regression function. the value of  $\varepsilon$  controls how many support vectors u get: larger  $\varepsilon$  = fewer support vectors = sparser model = faster prediction.

### 3.3 Choosing $\varepsilon$ and $C$

- $\varepsilon$ : controls the width of the tube. typical values: 0.01 to 0.5, depending on the noise level in ur data. rule of thumb: set  $\varepsilon$  to be slightly larger than the expected noise standard deviation.
- $C$ : regularization parameter. large  $C$  = less regularization = fit the data more tightly (might overfit). small  $C$  = more regularization = smoother function (might underfit).
- for quantum kernels, u typically want moderate  $C$  (1 to 100) bcs the kernel itself provides implicit regularization through its structure.

## 4 Pegasos QSVC

standard SVM solvers (like libsvm used by sklearn) solve the dual problem exactly, which requires storing and manipulating the full  $M \times M$  kernel matrix. for quantum kernels, this means computing all  $O(M^2)$  kernel entries upfront.

the pegasos algorithm offers an alternative: stochastic gradient descent on the primal SVM objective, which only needs kernel evaluations for sampled data points at each step.

#### 4.1 The Pegasos Algorithm

Definition 4.1 (Pegasos (Primal Estimated sub-GrADient SOLver for SVM)). shalev-shwartz et al. (2011) proposed the following online algorithm:

initialize  $\mathbf{w}_1 = \mathbf{0}$ , step size  $\eta_t = \frac{1}{\lambda t}$

for  $t = 1, 2, \dots, T$ :

1. sample a mini-batch  $A_t \subset \{1, \dots, M\}$  of size  $k$

2. compute the subgradient:

$$A_t^+ = \{i \in A_t : y_i(\mathbf{w}_t \cdot \phi(\mathbf{x}_i)) < 1\}$$

3. update:

$$\mathbf{w}_{t+1} = (1 - \eta_t \lambda) \mathbf{w}_t + \frac{\eta_t}{|A_t|} \sum_{i \in A_t^+} y_i \phi(\mathbf{x}_i)$$

4. project:  $\mathbf{w}_{t+1} = \min\left(1, \frac{1/\sqrt{\lambda}}{\|\mathbf{w}_{t+1}\|}\right) \mathbf{w}_{t+1}$

in kernel form, the weight vector is represented as:

$$\mathbf{w}_t = \sum_{i=1}^M \beta_i^{(t)} \phi(\mathbf{x}_i)$$

so the decision function  $\mathbf{w}_t \cdot \phi(\mathbf{x}) = \sum_i \beta_i^{(t)} \kappa(\mathbf{x}_i, \mathbf{x})$  only requires kernel evaluations.

#### 4.2 Pegasos vs Standard SVM

| Property            | Standard SVM (libsvm)      | Pegasos                              |
|---------------------|----------------------------|--------------------------------------|
| kernel evaluations  | $O(M^2)$ upfront           | $O(kT)$ over training                |
| memory              | $O(M^2)$ for kernel matrix | $O(M)$ for coefficients              |
| solution quality    | exact (up to solver tol)   | approximate                          |
| convergence         | finite (QP solver)         | $O(1/(\lambda \epsilon))$ iterations |
| mini-batch friendly | no                         | yes                                  |

Key Point. the key advantage of pegasos for quantum kernels: u dont need to compute the full  $M \times M$  kernel matrix. at each iteration, u only need kernel evaluations for the mini-batch. if the mini-batch size is  $k$  and u do  $T$  iterations, u need  $O(kTM)$  kernel evaluations in the worst case (if all training points become part of the representation). in practice, its much less.

#### 4.3 When to Use Pegasos QSVC

- larger datasets ( $M > 200$ ): the  $O(M^2)$  cost of full kernel matrix becomes painful. pegasos lets u train with a “budget” of kernel evaluations.
- online/streaming settings: new data arrives over time. pegasos naturally handles this without recomputing the full kernel matrix.

- when approximate is good enough: pegasos converges to a solution thats close to optimal. for noisy quantum kernels, the kernel matrix itself is approximate, so an approximate solver is fine.
- not recommended when:  $M$  is small ( $< 100$ ), u need the exact SVM solution, or the kernel is cheap to compute (classical kernels).

#### 4.4 Convergence Analysis

Theorem 4.1 (Pegasos Convergence). after  $T$  iterations with mini-batch size  $k = 1$ , pegasos achieves:

$$\mathbb{E}[f(\mathbf{w}_T)] - f(\mathbf{w}^*) \leq \frac{c}{\lambda T}$$

where  $f$  is the SVM objective,  $\mathbf{w}^*$  is the optimal solution, and  $c$  is a constant depending on the kernel bound  $\max_i \kappa(\mathbf{x}_i, \mathbf{x}_i) \leq R^2$ .

for quantum kernels,  $\kappa(\mathbf{x}_i, \mathbf{x}_i) = 1$  always (fidelity of a state with itself), so  $R^2 = 1$ .

this means after  $T = \frac{c}{\lambda \varepsilon}$  iterations, we have  $\varepsilon$ -optimal solution. with  $\lambda = 1/C$ :

$$T = O\left(\frac{C}{\varepsilon}\right)$$

for  $C = 10$  and  $\varepsilon = 0.01$ , thats about  $T = 1000$  iterations. with mini-batch size  $k = 5$ , thats 5000 kernel evaluations — much less than the  $M^2/2$  needed for the full matrix when  $M > 100$ .

### 5 Kernel Matrix Analysis

understanding ur kernel matrix is crucial. the eigenvalue spectrum tells u a lot about whether the kernel is useful.

#### 5.1 Eigenvalue Decomposition

the kernel matrix  $K \in \mathbb{R}^{M \times M}$  is symmetric positive semi-definite (in the ideal case). its eigendecomposition is:

$$K = \sum_{i=1}^M \lambda_i \mathbf{v}_i \mathbf{v}_i^\top, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_M \geq 0$$

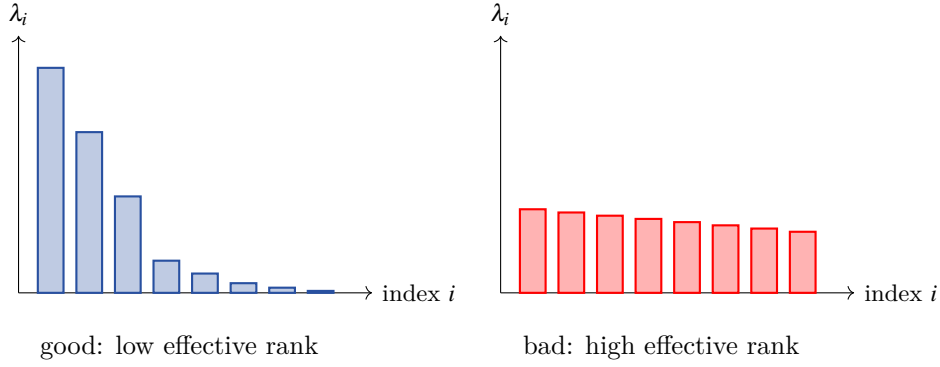
Definition 5.1 (Effective Rank). the effective rank of the kernel matrix is:

$$r_{\text{eff}}(K) = \exp\left(-\sum_{i=1}^M \frac{\lambda_i}{\text{tr}(K)} \log \frac{\lambda_i}{\text{tr}(K)}\right)$$

this is the exponential of the entropy of the normalized eigenvalue distribution. it ranges from 1 (one dominant eigenvalue) to  $M$  (all eigenvalues equal).



## 5.2 What the Spectrum Tells U



interpretation of different eigenvalue patterns:

1. few large eigenvalues, rapid decay ( $r_{\text{eff}} \ll M$ ): the kernel captures a few important directions. this is good — the SVM will find a good classifier in a low-dimensional effective subspace.
2. all eigenvalues similar ( $r_{\text{eff}} \approx M$ ): the kernel is essentially random. every direction is equally (un)important. this means the kernel doesn't encode useful structure. this often happens when:
  - too many qubits (exponential concentration)
  - feature map is not matched to the data structure
  - too much noise in the kernel estimates
3. one very large eigenvalue, rest small: the kernel is almost constant — all data points look similar. this means the feature map maps everything to roughly the same quantum state.
4. negative eigenvalues: the estimated kernel matrix is not PSD. this happens due to finite shot noise. small negative eigenvalues ( $|\lambda_{\min}| < 0.01$ ) are fine — just clip them. large negative eigenvalues indicate a problem.

## 5.3 Connection to SVM Performance

Theorem 5.1 (Eigenvalue Spectrum and Generalization). the generalization error of the SVM trained with kernel  $K$  is related to the eigenvalue spectrum through:

$$\text{expected error} \leq O\left(\frac{1}{M} \sum_{i=1}^M \min\left(\frac{1}{C\lambda_i}, 1\right)\right)$$

this bound is tightest when a few eigenvalues are large (corresponding to the useful features) and the rest are small (noise directions that get regularized away).

Intuition. think of eigenvalues as “feature importances” in the quantum feature space. large eigenvalues correspond to directions where data points vary a lot — these are the useful features. small eigenvalues correspond to noise directions. the SVM naturally focuses on the large-eigenvalue directions (bcs they have more signal), so a kernel with a few large eigenvalues is ideal.

## 6 Training on Real Datasets

now lets get serious and look at training quantum kernel SVMs on actual (well, standard ML benchmark) datasets.

### 6.1 Iris Dataset

dataset: 150 samples, 4 features, 3 classes (setosa, versicolor, virginica).

preprocessing:

1. select 2 classes for binary classification (versicolor vs virginica — the harder pair)
2. select 2 features (petal length and width — the most discriminative)
3. standard scaling then rescale to  $[0, \pi]$
4. 70/30 train/test split, stratified

quantum setup: 2 qubits, ZZFeatureMap, reps=2, full entanglement, 1024 shots.

approach: compute the full kernel matrix for training data ( $\sim 70 \times 70/2 \approx 2450$  circuits). train QSVC with  $C = 1.0$ . compute test kernel matrix ( $\sim 30 \times 70 = 2100$  circuits). predict on test set.

expected results:

- training accuracy:  $\sim 93\text{--}96\%$
- test accuracy:  $\sim 90\text{--}95\%$
- number of support vectors:  $\sim 15\text{--}25$
- kernel alignment:  $\sim 0.15\text{--}0.25$

comparison: classical RBF kernel with tuned  $\gamma$  typically gets 95–97% test accuracy. so quantum is close but slightly below.

### 6.2 Wine Dataset

dataset: 178 samples, 13 features, 3 classes.

preprocessing: this is where it gets tricky — 13 features but we can only use  $n$  qubits.

1. select 2 classes (class 1 vs class 2)
2. apply PCA to reduce to 4 features (4 qubits)
3. standard scaling then rescale to  $[0, \pi]$
4. 70/30 train/test split

quantum setup: 4 qubits, ZZFeatureMap, reps=2, linear entanglement, 2048 shots.

why linear entanglement: with 4 qubits, full entanglement gives  $\binom{4}{2} = 6$  ZZ interactions per layer. the circuit depth becomes  $\sim 40$  gates per rep, so the kernel circuit has  $\sim 160$  gates. with linear entanglement (3 ZZ interactions per layer), depth drops to  $\sim 100$  gates. on noisy hardware, this matters a lot.

expected results:

- training accuracy:  $\sim 90\text{--}95\%$
- test accuracy:  $\sim 85\text{--}92\%$
- kernel alignment:  $\sim 0.10\text{--}0.20$

Intuition. the wine dataset is harder for quantum kernels bcs the PCA step throws away information that might be in the discarded principal components. classical kernels operating on all 13 features have an inherent advantage here. this illustrates a fundamental limitation: quantum kernels r constrained by the number of available qubits.

### 6.3 Breast Cancer Dataset

dataset: 569 samples, 30 features, 2 classes (malignant vs benign).

preprocessing:

1. PCA to 4 features (explaining  $\sim 95\%$  of variance)
2. standard scaling then rescale to  $[0, \pi]$
3. 70/30 train/test split, stratified

quantum setup: 4 qubits, ZZFeatureMap, reps=2, 2048 shots.

expected results:

- training accuracy:  $\sim 92\text{--}96\%$
- test accuracy:  $\sim 90\text{--}95\%$
- kernel alignment:  $\sim 0.20\text{--}0.30$

computational cost:  $M_{\text{train}} \approx 400$ , so we need  $\sim 80,000$  circuits for the training kernel matrix. at 2048 shots each, thats  $\sim 164$  million circuit shots. on a simulator, this takes minutes. on real hardware, this is a significant job.

Key Point. with 569 samples, the breast cancer dataset pushes the limits of quantum kernel methods. the  $O(M^2)$  scaling means  $\sim 160,000$  kernel evaluations. this is where pegasos QSVC becomes attractive — it can train with a fraction of the kernel evaluations.

## 7 Cross-Validation with Quantum Kernels

cross-validation is essential for honest performance evaluation, but its expensive with quantum kernels.

### 7.1 The Cost Problem

for  $k$ -fold cross-validation:

- $k$  different train/test splits
- each split requires a new kernel matrix (different training set  $\rightarrow$  different kernel entries needed)

- naive approach:  $k$  full kernel matrix computations

total cost without optimization:

$$\text{cost}_{\text{naive}} = k \cdot \frac{M_{\text{train}}(M_{\text{train}} - 1)}{2} \cdot N_s$$

for  $k = 5$ ,  $M = 100$ ,  $N_s = 1024$ : thats  $5 \times 4950 \times 1024 \approx 25$  million circuit shots.

## 7.2 Smart Cross-Validation

Key Point. the key insight: compute the full  $M \times M$  kernel matrix once using all data points. then for each fold, extract the relevant submatrix. this works bcs  $\kappa(\mathbf{x}_i, \mathbf{x}_j)$  doesnt depend on which fold ur in — the kernel value between two points is always the same.

the procedure:

1. compute the full kernel matrix  $K \in \mathbb{R}^{M \times M}$  — cost:  $\frac{M(M-1)}{2} \cdot N_s$
2. for each fold  $f = 1, \dots, k$ :
  - let  $\mathcal{J}_f^{\text{train}}$  and  $\mathcal{J}_f^{\text{test}}$  be the train and test indices
  - extract  $K_{\text{train}}^{(f)} = K[\mathcal{J}_f^{\text{train}}, \mathcal{J}_f^{\text{train}}]$
  - extract  $K_{\text{test}}^{(f)} = K[\mathcal{J}_f^{\text{test}}, \mathcal{J}_f^{\text{train}}]$
  - train SVM with  $K_{\text{train}}^{(f)}$  (this is fast — classical computation)
  - predict using  $K_{\text{test}}^{(f)}$
3. average the scores across folds

total cost:

$$\text{cost}_{\text{smart}} = \frac{M(M-1)}{2} \cdot N_s + k \cdot (\text{classical SVM cost})$$

the classical SVM cost is negligible compared to the quantum kernel computation. so smart cross-validation is essentially free once u have the full kernel matrix.

## 7.3 Nested Cross-Validation

for honest hyperparameter tuning, u need nested cross-validation: an outer loop for performance estimation and an inner loop for hyperparameter selection.

the problem: if u change the feature map hyperparameters (reps, entanglement), u need a new kernel matrix. so for  $p$  hyperparameter settings, u need  $p$  full kernel matrices.

$$\text{cost}_{\text{nested}} = p \cdot \frac{M(M-1)}{2} \cdot N_s \tag{10}$$

$$+ k_{\text{outer}} \cdot k_{\text{inner}} \cdot (\text{classical SVM cost}) \tag{11}$$

for  $p = 10$  hyperparameter settings,  $M = 100$ ,  $N_s = 1024$ : thats  $10 \times 4950 \times 1024 \approx 50$  million circuit shots. still feasible on a simulator, but getting expensive on real hardware.

## 8 Hyperparameter Tuning

quantum kernel SVMs have several hyperparameters to tune. lets go through them systematically.

### 8.1 Feature Map Parameters

1. feature map type: ZFeatureMap, ZZFeatureMap, PauliFeatureMap, or custom. start with ZZFeatureMap — its the standard choice with a good expressivity/cost tradeoff.
2. number of repetitions (reps): controls depth and expressivity.
  - reps=1: shallow, less expressive, more noise-robust
  - reps=2: good default, reasonable depth
  - reps=3+: deep, more expressive, noise-sensitiverecommendation: try  $\text{reps} \in \{1, 2, 3\}$  and pick the best by cross-validation.
3. entanglement scheme: full, linear, circular, or sca.
  - for  $n \leq 4$  qubits: full is fine
  - for  $n \geq 5$  qubits: linear or circular to keep depth manageable
4. data encoding function: the default is  $\phi_S(\mathbf{x}) = \prod_{j \in S} x_j$ . alternatives include  $\phi_S(\mathbf{x}) = \prod_{j \in S} (\pi - x_j)$  or polynomial functions of  $x_j$ .

### 8.2 SVM Hyperparameters

1.  $C$  (regularization): the penalty for margin violations.
  - small  $C$  (0.01–0.1): heavy regularization, smoother boundary, fewer support vectors
  - moderate  $C$  (1–10): balanced
  - large  $C$  (100+): little regularization, complex boundary, many support vectorstypical grid:  $C \in \{0.1, 1, 10, 100\}$ .
2.  $\epsilon$  (for QSVR): tube width. typical grid:  $\epsilon \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$ .

### 8.3 Tuning Strategy

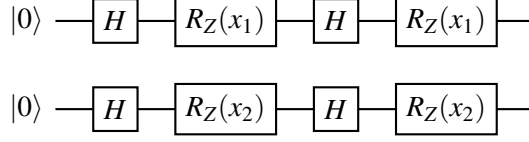
Key Point. the most efficient tuning strategy for quantum kernels:

1. first, use kernel alignment (cheap to compute) to narrow down feature map choices. compute alignment for each candidate feature map and discard those with alignment  $< 0.05$ .
2. then, for the surviving feature maps, do cross-validation to tune  $C$  (and  $\epsilon$  for regression).
3. finally, evaluate the best configuration on a held-out test set.

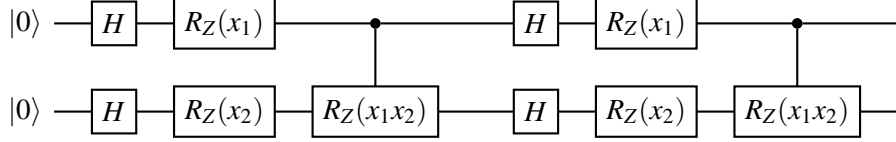
this saves a lot of quantum computation bcs alignment doesnt require SVM training.

the quantikz circuits for comparing different feature maps:

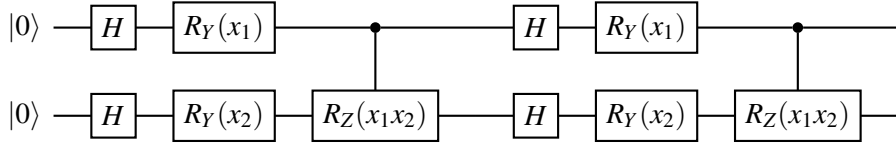
ZFeatureMap (no entanglement):



ZZFeatureMap (pairwise entanglement):



PauliFeatureMap with Y and ZZ:



note how the ZFeatureMap has no two-qubit gates, while ZZFeatureMap and PauliFeatureMap include controlled rotations that create entanglement dependent on the data.

## 9 Computational Cost Analysis

lets be rigorous about the computational costs. this is important bcs quantum kernels r often presented as “working” without mentioning how expensive they r.

### 9.1 Kernel Matrix Construction

Definition 9.1 (Quantum Kernel Computation Cost). for  $M$  data points,  $n$  qubits, feature map depth  $D$  (in gates), and  $N_s$  shots:

$$\text{unique circuits} = \frac{M(M-1)}{2} \quad (12)$$

$$\text{gates per circuit} = 2D \quad (\text{compute-uncompute}) \quad (13)$$

$$\text{total shots} = \frac{M(M-1)}{2} \cdot N_s \quad (14)$$

$$\text{total gate operations} = \frac{M(M-1)}{2} \cdot N_s \cdot 2D \quad (15)$$

lets put real numbers on this:

| $M$  | $N_s$ | circuits | total shots | time (sim)          |
|------|-------|----------|-------------|---------------------|
| 50   | 1024  | 1,225    | 1.25M       | $\sim 10\text{s}$   |
| 100  | 1024  | 4,950    | 5.07M       | $\sim 1\text{min}$  |
| 200  | 1024  | 19,900   | 20.4M       | $\sim 5\text{min}$  |
| 500  | 1024  | 124,750  | 127.7M      | $\sim 30\text{min}$ |
| 1000 | 1024  | 499,500  | 511.5M      | $\sim 2\text{hrs}$  |

(times r rough estimates for a statevector simulator on a modern laptop. actual hardware times depend on queue and calibration.)

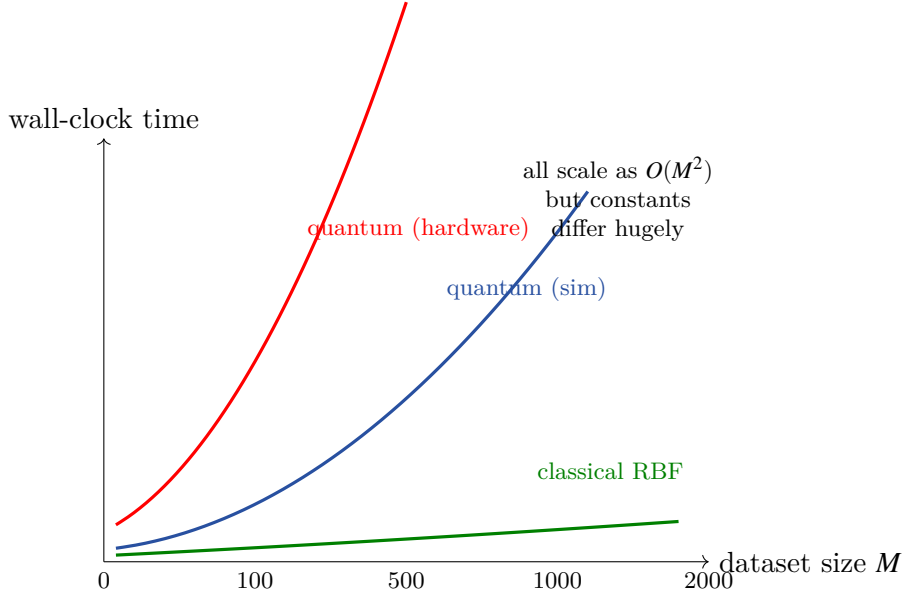
## 9.2 Prediction Cost

for a new test point, u need  $|\mathcal{SV}|$  kernel evaluations (one per support vector). each requires  $N_s$  shots.

$$\text{prediction cost per point} = |\mathcal{SV}| \cdot N_s \cdot 2D \text{ gate operations}$$

if the SVM has 30 support vectors and  $N_s = 1024$ : thats 30,720 circuit shots per prediction. not terrible, but not instant either.

## 9.3 Comparison with Classical Kernels



the key point: quantum and classical kernels both scale as  $O(M^2)$  for training, but the constant factor is enormously different. computing  $\kappa_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}$  takes nanoseconds. computing  $\kappa_{\text{quantum}}(\mathbf{x}, \mathbf{x}')$  takes microseconds (simulator) to milliseconds (hardware per shot). thats a factor of  $10^3$  to  $10^6$ .

Key Point. quantum kernels can only justify their computational cost if they provide a classification/regression quality that classical kernels cannot match, not just one that they happen not to match with default hyperparameters. this is a very high bar.

## 9.4 Total Budget Planning

for a practical quantum kernel experiment, plan ur compute budget:

$$\text{total budget} = \underbrace{p \cdot \frac{M(M-1)}{2} \cdot N_s}_{\text{hyperparameter search}} + \underbrace{\frac{M(M-1)}{2} \cdot N_s}_{\text{final training}} + \underbrace{M_{\text{test}} \cdot |\mathcal{SV}| \cdot N_s}_{\text{prediction}} \quad (16)$$

where  $p$  is the number of hyperparameter configurations to try. for  $p = 12$  ( $3 \text{ reps} \times 4 \text{ } C \text{ values}$ ),  $M = 100$ ,  $N_s = 1024$ :

$$\text{budget} = 12 \times 4950 \times 1024 + 4950 \times 1024 + 30 \times 20 \times 1024 \approx 66 \text{ million shots}$$

## 10 Classical vs Quantum Kernel Comparison

when does quantum actually win? lets be precise.

### 10.1 When Quantum Kernels Can Win

Theorem 10.1 (Quantum Advantage for Kernels (Liu, Arunachalam, Temme 2021)). there exist classification problems where:

1. a quantum kernel achieves low prediction error
2. any classical kernel (with polynomial-time computable entries) achieves high prediction error

the separation is based on the hardness of the discrete logarithm problem and requires data with specific structure related to quantum computation.

Intuition. the proven quantum advantage is for constructed problems where the data is specifically designed to be easy for quantum kernels. these r not natural ML problems — theyre cryptographic in nature. for natural datasets, no proven advantage exists (yet).

### 10.2 Empirical Comparison: Feature Map Expressivity

on standard ML benchmarks, here is the typical picture:

| Kernel                  | Iris | Wine | Cancer | Compute   |
|-------------------------|------|------|--------|-----------|
| linear                  | 87%  | 95%  | 95%    | fast      |
| RBF (tuned)             | 95%  | 98%  | 97%    | fast      |
| polynomial (tuned)      | 93%  | 97%  | 96%    | fast      |
| quantum ZFeatureMap     | 88%  | 90%  | 92%    | slow      |
| quantum ZZFeatureMap    | 93%  | 93%  | 94%    | slow      |
| quantum PauliFeatureMap | 94%  | 94%  | 95%    | very slow |

Key Point. on standard benchmark datasets, quantum kernels perform comparably to classical kernels but r orders of magnitude slower. the quantum kernels with entanglement (ZZFeatureMap, PauliFeatureMap) consistently outperform those without (ZFeatureMap), confirming that entanglement adds expressivity. but classical RBF with tuned  $\gamma$  is very hard to beat.

### 10.3 When Quantum Doesnt Win

be honest about the limitations:



1. smooth low-dimensional data: if the decision boundary is smooth and the data lives in low dimensions, RBF kernel captures it perfectly. quantum kernels cant do better bcs theres nothing more to capture.
2. high-dimensional data with linear structure: if the data is linearly separable in the original space (or after a simple feature engineering step), a linear kernel suffices. adding quantum complexity is wasteful.
3. very noisy data: if the bayes-optimal error is high (say  $> 20\%$ ), no kernel (quantum or classical) will do well. the noise floor is a fundamental limit.
4. large datasets: the  $O(M^2)$  cost makes quantum kernels impractical for  $M > 1000$  on current hardware.

## 11 Limitations and When NOT to Use Quantum Kernels

this section is arguably the most important in the whole lecture. knowing when not to use a method is just as valuable as knowing when to use it.

### 11.1 Dataset Size Limits

the  $O(M^2)$  kernel matrix is the fundamental bottleneck:

| $M$    | kernel entries | at 1024 shots | feasibility             |
|--------|----------------|---------------|-------------------------|
| 50     | 1,225          | 1.25M shots   | easy                    |
| 100    | 4,950          | 5.07M shots   | comfortable             |
| 500    | 124,750        | 127.7M shots  | challenging             |
| 1,000  | 499,500        | 511.5M shots  | hard                    |
| 10,000 | 49,995,000     | 51.2B shots   | impossible (current hw) |

Key Point. practical limit with current quantum hardware:  $M \leq 200-500$  depending on the number of qubits and shots. with simulators, u can push to  $M \sim 1000$ , but the classical simulation itself becomes slow for  $n > 20$  qubits.

### 11.2 Noise Sensitivity

recall from last lecture: noise pushes all kernel values toward  $1/2^n$ . the signal-to-noise ratio of the kernel:

$$\text{SNR} = \frac{|\kappa_{\text{ideal}}(\mathbf{x}, \mathbf{x}') - 1/2^n|}{1/\sqrt{N_s}} \cdot (1-p)^{2D_{\text{gates}}}$$

for the kernel to be useful, u need  $\text{SNR} \gg 1$ . this constrains:

- number of qubits  $n$  (bcs of  $1/2^n$  noise floor)
- circuit depth  $D_{\text{gates}}$  (bcs of exponential fidelity decay)
- number of shots  $N_s$  (bcs of statistical noise)

### 11.3 The Concentration Problem

Definition 11.1 (Kernel Concentration). a kernel  $\kappa$  exhibits concentration if for random data points  $\mathbf{x}, \mathbf{x}'$ :

$$\text{Var}_{\mathbf{x}, \mathbf{x}'}[\kappa(\mathbf{x}, \mathbf{x}')] \rightarrow 0 \text{ as } n \rightarrow \infty$$

meaning all off-diagonal kernel entries converge to the same value. the kernel becomes uninformative.

Theorem 11.1 (Exponential Concentration (Thanasilp et al., 2022)). for the standard quantum kernel with  $n$  qubits and feature maps forming approximate 2-designs:

$$\text{Var}_{\mathbf{x}, \mathbf{x}'}[\kappa(\mathbf{x}, \mathbf{x}')] \leq O\left(\frac{1}{2^n}\right)$$

the kernel concentrates exponentially in the number of qubits.

Intuition. this is the quantum kernel version of the “curse of dimensionality.” as u add more qubits, the hilbert space grows as  $2^n$ , and random quantum states become nearly orthogonal. the kernel value between any two random points approaches  $1/2^n \approx 0$ , and the variance around this value shrinks exponentially. the kernel matrix becomes  $K \approx I$  (identity), which is useless for classification.

ways to mitigate concentration:

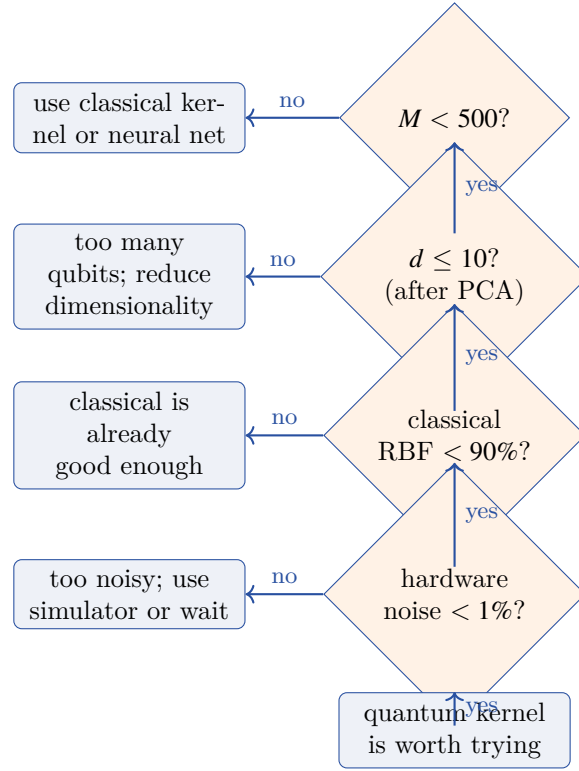
1. use fewer qubits (match to the intrinsic dimensionality of the data)
2. use projected quantum kernels (measure local observables instead of global fidelity)
3. use shallow circuits with limited entanglement
4. use data-dependent feature maps trained to avoid concentration

### 11.4 When NOT to Use Quantum Kernels

a definitive list:

1. dataset has  $> 1000$  samples: the  $O(M^2)$  cost is prohibitive. use a classical kernel SVM, random features approximation, or a neural network instead.
2. data is high-dimensional and u need many qubits: if  $n > 10$  qubits, concentration will likely kill performance. either reduce dimensionality aggressively or use a different approach.
3. classical RBF kernel already works well: if u can get  $> 95\%$  accuracy with a tuned RBF kernel, adding quantum complexity wont help (and will cost orders of magnitude more compute).
4. running on very noisy hardware: if gate error rates  $r$  above 1% and ur circuit has  $> 50$  gates, the kernel signal will be drowned in noise.
5. real-time prediction is needed: each prediction requires  $|\mathcal{SV}|$  quantum circuit evaluations. this is too slow for real-time applications.
6. interpretability is important: quantum kernels r black boxes on top of black boxes. if u need to explain why the model makes a decision, use a simpler model.

## 11.5 Decision Flowchart



## 12 Advanced Topics

### 12.1 Trainable Quantum Kernels

instead of using a fixed feature map, u can add trainable parameters:

$$U(\mathbf{x}; \theta) = V(\theta)W(\mathbf{x})V'(\theta)$$

where  $V(\theta)$  and  $V'(\theta)$  r trainable unitaries and  $W(\mathbf{x})$  is the data encoding. the parameters  $\theta$  r optimized to maximize kernel alignment:

$$\theta^* = \arg \max_{\theta} A(K(\theta), \mathbf{y}\mathbf{y}^\top)$$

this is a quantum-classical optimization loop:

1. initialize  $\theta$  randomly
2. compute kernel matrix  $K(\theta)$
3. compute alignment  $A$  and its gradient w.r.t.  $\theta$
4. update  $\theta$  using gradient ascent
5. repeat until convergence

the gradient of each kernel entry w.r.t.  $\theta_k$  can be computed using the parameter shift rule:

$$\frac{\partial K_{ij}}{\partial \theta_k} = \frac{K_{ij}(\theta_k + \pi/2) - K_{ij}(\theta_k - \pi/2)}{2}$$

this requires computing two additional kernel matrices per parameter per gradient step. expensive, but can significantly improve kernel quality.

## 12.2 Quantum Kernel Bandwidth

by analogy with the RBF kernel ( $k(\mathbf{x}, \mathbf{x}') = e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}$ ), quantum kernels have an implicit “bandwidth” controlled by the feature map scaling.

if u encode data as  $U(\alpha \mathbf{x})$  where  $\alpha$  is a scaling parameter:

- small  $\alpha$ : all quantum states r similar  $\rightarrow$  kernel values close to 1  $\rightarrow$  underfitting
- large  $\alpha$ : all quantum states r different  $\rightarrow$  kernel values close to 0  $\rightarrow$  overfitting
- optimal  $\alpha$ : depends on the data distribution

Key Point. the data rescaling factor  $\alpha$  acts like an inverse bandwidth for quantum kernels. tuning it is at least as important as tuning the feature map architecture. a good starting point is  $\alpha = 1$  (data in  $[0, \pi]$ ), then search over  $\alpha \in \{0.5, 1, 2, 4\}$ .

## 12.3 Kernel Principal Component Analysis

u can use the quantum kernel matrix for dimensionality reduction via kernel PCA:

1. compute the  $M \times M$  quantum kernel matrix  $K$
2. center the kernel:  $\tilde{K} = K - \frac{1}{M} \mathbf{1}\mathbf{1}^\top K - K \frac{1}{M} \mathbf{1}\mathbf{1}^\top + \frac{1}{M^2} \mathbf{1}\mathbf{1}^\top K \mathbf{1}\mathbf{1}^\top$
3. eigendecompose:  $\tilde{K} = V \Lambda V^\top$
4. project: new features r the columns of  $V \Lambda^{1/2}$  (top  $k$  components)

the quantum kernel PCA features can then be used with any classical ML algorithm, not just SVMs. this is a nice way to extract value from quantum kernels without committing to the SVM framework.

## 13 Putting It All Together

heres the recommended workflow for a quantum kernel project:

1. baseline first: train a classical SVM with RBF kernel, tune  $\gamma$  and  $C$ . this is ur target to beat.
2. feasibility check: is  $M < 500$ ? is  $d$  (after PCA)  $\leq 10$ ? if no, dont bother with quantum kernels.
3. feature map selection: compute kernel alignment for several feature maps (ZZFeatureMap with reps=1,2,3 and different entanglements). pick the top 2-3 by alignment.
4. cross-validation: for each selected feature map, compute the full kernel matrix and do 5-fold CV over  $C \in \{0.1, 1, 10, 100\}$ .
5. compare: is the best quantum kernel within  $\pm 2\%$  of the classical baseline? if not, the classical kernel is better. if yes, report that quantum is “competitive” (not “better”).

6. analyze: look at the kernel matrix structure, eigenvalue spectrum, number of support vectors. understand why the kernel works or doesn't.
7. report honestly: include computational cost, noise analysis (if on real hardware), and comparison with classical baselines. don't cherry-pick.

## 14 Summary and Key Takeaways

this lecture covered the practical and theoretical aspects of using quantum kernels with SVMs. the main points:

1. QSVC and QSVR r straightforward: plug a quantum kernel into a classical SVM. the math is identical to classical kernel SVMs — the only difference is how the kernel entries r computed.
2. pegasos QSVC offers a way to train without the full  $O(M^2)$  kernel matrix. useful for larger datasets ( $M > 200$ ) where computing all kernel entries is expensive.
3. eigenvalue analysis of the kernel matrix tells u whether the kernel is informative. look for rapid eigenvalue decay (low effective rank) — that's the sign of a useful kernel.
4. cross-validation is essential but can be done efficiently by computing the full kernel matrix once and extracting submatrices for each fold.
5. hyperparameter tuning: use alignment-based prefiltering, then cross-validation for  $C$  (and  $\epsilon$ ). don't forget the data scaling factor  $\alpha$ .
6. computational cost is the elephant in the room: quantum kernels r  $10^3$ – $10^6$  times slower per kernel entry than classical kernels. this cost is only justified if the quantum kernel provides genuinely better classification.
7. on standard benchmarks, classical wins or ties: this is the honest conclusion from current research. quantum advantage for kernels has been proven to exist for constructed problems, but not demonstrated for natural ML problems.
8. know when NOT to use quantum kernels: large datasets, high-dimensional data, noisy hardware, or when classical already works well. this is perhaps the most important lesson.
9. the concentration problem is fundamental: more qubits  $\neq$  better kernels. match the number of qubits to the intrinsic dimensionality of ur data.
10. honest benchmarking: always compare against tuned classical baselines, report computational costs, use proper cross-validation, and don't cherry-pick results.

next lecture we'll move beyond kernels entirely and look at quantum neural networks and variational classifiers — a completely different approach to quantum machine learning.